

UNIVERSIDADE FEDERAL DO RIO DE JANEIRO

INSTITUTO DE MATEMÁTICA

DEPARTAMENTO DE MATEMÁTICA

DISCIPLINA: ESTRUTURA DE DADOS (ED)

Docente: Prof. Milton R. Ramirez

`milton@matematica.ufrj.br`

TRABALHO PRÁTICO: IMPLEMENTAÇÃO DA CLASSE PILHA

Implementações nas linguagens

Python, C/C++ e JavaScript

Integrantes do grupo:

Emanuel Cardoso da Silva	122033151
Alanna Figueiredo Simões	120053919
Felipe Rodrigues Schoeffer	123311188
Artur de Melo Brito	117224234
Matheus de Sousa	123536584

Rio de Janeiro

2026

Sumário

1	Introdução	4
2	Objetivos	5
2.1	Objetivo geral	5
2.2	Objetivos específicos	5
3	Especificação da Classe Pilha	7
3.1	Conceito	7
3.2	Capacidade	8
3.3	Homogeneidade de tipos	8
3.4	Operações	8
3.5	Tratamento de erros	8
4	Implementação em Python	10
4.1	Descrição	10
4.2	Implementação	10
4.3	Construtor	12
4.4	Operação empilha	13
4.5	Operação desempilha	13
4.6	Testes em Python	13
4.7	Resultado	14
5	Implementação em C/C++	16
5.1	Descrição	16
5.2	Implementação	16
5.3	Array	20

5.4	Destrutor	20
5.5	Operação empilha	20
5.6	Operação desempilha	21
5.7	Testes em C++	21
6	Implementação em JavaScript	24
6.1	Descrição	24
6.2	Implementação	24
6.3	TypedArray	29
6.4	Verificação de tipos	30
6.5	Testes em JavaScript	30
7	Testes e Debug	33
7.1	Estratégia de testes	33
7.2	Teste do comportamento LIFO	33
7.3	Debug	34
7.4	Teste de pilha cheia	34
7.5	Teste de tipo	35
8	Code Review	36
8.1	Código inicial	36
8.2	Problemas identificados	36
8.3	Uso inadequado de lista	36
8.4	Verificação de tipos	37
8.5	Importância do Code Review	37
9	Análise de Complexidade	38
9.1	Inserção	38
9.2	Remoção	38
9.3	Verificação de vazio	38
9.4	Verificação de cheio	39
9.5	Tamanho	39
9.6	Troca	39

9.7	Complexidade espacial	39
9.8	Resumo	39
10	Comparação entre as Implementações	41
10.1	Python	41
10.2	C/C++	41
10.3	JavaScript	42
11	Testes de Estresse	43
12	Post-Mortem	44
12.1	Objetivo	44
12.2	Uso da Inteligência Artificial	44
12.3	Problemas encontrados	44
12.4	Correções realizadas	45
12.5	Aprendizados	45
12.6	Resultado final	45
13	Conclusão	46

Capítulo 1

Introdução

Estruturas de dados são recursos fundamentais da computação para organizar e manipular informações de maneira eficiente. Entre as estruturas lineares mais importantes encontra-se a pilha, conhecida também pelo termo em inglês *stack*.

A pilha possui como principal característica o princípio LIFO (*Last In, First Out*), segundo o qual o último elemento inserido é o primeiro elemento removido.

O presente trabalho tem como objetivo desenvolver uma classe **Pilha** utilizando três linguagens de programação distintas: Python, C/C++ e JavaScript.

A implementação deve respeitar uma especificação comum, possibilitando comparar como diferentes linguagens tratam aspectos como armazenamento, tipos de dados, tratamento de erros e gerenciamento de memória.

A estrutura desenvolvida deve aceitar apenas elementos de um mesmo tipo básico por instância. Os tipos considerados são:

- caractere;
- inteiro;
- ponto flutuante.

Além disso, a pilha possui capacidade máxima previamente determinada. Quando essa capacidade é atingida, novas inserções devem gerar um erro.

O trabalho também contempla a realização de testes e procedimentos de debug, permitindo verificar se as implementações atendem aos requisitos estabelecidos.

Por fim, é realizada uma análise crítica do desenvolvimento assistido por Inteligência Artificial, incluindo Code Review, identificação de problemas e Post-Mortem.

Capítulo 2

Objetivos

2.1 Objetivo geral

Implementar uma estrutura de dados do tipo pilha por meio de uma classe orientada a objetos, utilizando as linguagens Python, C/C++ e JavaScript, respeitando os requisitos de capacidade, homogeneidade de tipos, operações e tratamento de erros.

2.2 Objetivos específicos

Os objetivos específicos são:

- implementar a classe `Pilha`;
- utilizar armazenamento baseado em array;
- definir uma capacidade máxima;
- controlar o elemento localizado no topo;
- implementar a operação de inserção;
- implementar a operação de remoção;
- verificar se a pilha está vazia;
- verificar se a pilha está cheia;
- obter o tamanho atual da pilha;
- implementar a operação de troca;
- impedir a inserção de tipos incompatíveis;

- tratar situações de erro;
- realizar testes das operações;
- realizar debug das implementações;
- analisar a complexidade dos algoritmos;
- comparar as três linguagens;
- analisar criticamente o código produzido com auxílio de IA.

Capítulo 3

Especificação da Classe Pilha

3.1 Conceito

Uma pilha é uma estrutura de dados linear na qual as operações de inserção e remoção ocorrem em uma única extremidade, denominada topo.

O comportamento segue o princípio:

LIFO = Last In, First Out.

Considere uma pilha inicialmente vazia:

[]

Ao inserir os elementos 10, 20 e 30, obtemos:

[10, 20, 30].

O elemento 30 está no topo. Portanto, a primeira remoção deve retornar 30.

Depois da remoção:

[10, 20].

A próxima remoção retorna 20.

Dessa forma, a ordem de remoção é:

30, 20, 10.

3.2 Capacidade

A pilha possui uma capacidade máxima definida no momento de sua criação.

Se a capacidade for:

$$C = 3,$$

somente três elementos podem ser armazenados simultaneamente.

Quando:

$$topo = C,$$

a estrutura está cheia e uma nova inserção deve produzir `PilhaCheiaErro`.

3.3 Homogeneidade de tipos

Cada instância da pilha deve aceitar apenas elementos de um mesmo tipo.

Por exemplo, uma pilha configurada para inteiros deve aceitar:

$$1, 2, 3, 100$$

mas não deve aceitar:

`"texto"`

ou um valor de ponto flutuante.

Essa regra garante que a estrutura seja homogênea.

3.4 Operações

As operações principais são apresentadas na Tabela 3.1.

3.5 Tratamento de erros

Dois erros são fundamentais na especificação.

O primeiro ocorre quando a pilha está cheia e o programa tenta inserir um novo

Operação	Função
<code>empilha(dado)</code>	Insere um elemento no topo.
<code>desempilha()</code>	Remove e retorna o elemento do topo.
<code>pilha_esta_vazia()</code>	Verifica se a pilha está vazia.
<code>pilha_esta_cheia()</code>	Verifica se a pilha está cheia.
<code>tamanho()</code>	Retorna a quantidade de elementos.
<code>troca()</code>	Troca os dois elementos do topo.

Tabela 3.1: Operações da classe Pilha.

elemento:

```
1 PilhaCheiaErro
```

O segundo ocorre quando o elemento inserido possui um tipo diferente do tipo definido para a pilha:

```
1 TipoErro
```

Também deve existir tratamento para a tentativa de remover um elemento de uma pilha vazia.

Capítulo 4

Implementação em Python

4.1 Descrição

A implementação em Python utiliza o módulo padrão `array` para representar o armazenamento interno da pilha.

Essa escolha permite utilizar uma estrutura de dados tipada em vez de simplesmente utilizar uma lista dinâmica.

A classe mantém três informações principais:

- capacidade;
- tipo permitido;
- posição do topo.

4.2 Implementação

```
1 from array import array
2
3
4 class PilhaCheiaErro(Exception):
5     pass
6
7
8 class TipoErro(Exception):
9     pass
10
11
```

```

12 class Pilha:
13     def __init__(self, capacidade, tipo):
14         self.capacidade = capacidade
15         self.tipo = tipo
16         self.topo = 0
17
18         if tipo == int:
19             self.dados = array('i', [0]) * capacidade
20
21         elif tipo == float:
22             self.dados = array('d', [0.0]) * capacidade
23
24         elif tipo == str:
25             self.dados = array('u', ['\0']) * capacidade
26
27         else:
28             raise TipoErro("Tipo n o suportado.")
29
30     def pilha_esta_vazia(self):
31         return self.topo == 0
32
33     def pilha_esta_cheia(self):
34         return self.topo == self.capacidade
35
36     def tamanho(self):
37         return self.topo
38
39     def empilha(self, dado):
40
41         if self.pilha_esta_cheia():
42             raise PilhaCheiaErro(
43                 "A pilha est cheia."
44             )
45
46         if not isinstance(dado, self.tipo):
47             raise TipoErro(
48                 "Tipo de dado incompat vel."
49             )
50
51         self.dados[self.topo] = dado
52         self.topo += 1

```

```

53
54     def desempilha(self):
55
56         if self.pilha_esta_vazia():
57             raise IndexError(
58                 "A pilha est vazia."
59             )
60
61         self.topo -= 1
62
63         dado = self.dados[self.topo]
64
65         return dado
66
67     def troca(self):
68
69         if self.topo < 2:
70             raise IndexError(
71                 "    necess rio possuir pelo menos dois elementos
72                 ."
73             )
74
75         self.dados[self.topo - 1], self.dados[self.topo - 2] = (
76             self.dados[self.topo - 2],
77             self.dados[self.topo - 1]
78         )

```

Listing 4.1: Classe Pilha implementada em Python.

4.3 Construtor

O construtor recebe:

```

1  capacidade
2  tipo

```

Por exemplo:

```

1  p = Pilha(5, int)

```

cria uma pilha com capacidade para cinco inteiros.

O atributo:

```
1 self.topo
```

começa com valor zero.

Isso significa que inicialmente não existe nenhum elemento armazenado.

4.4 Operação empilha

O método `empilha()` realiza duas verificações.

Primeiramente verifica se a pilha está cheia.

Depois verifica se o tipo do dado corresponde ao tipo da pilha.

Somente após essas verificações o elemento é armazenado.

O índice utilizado é:

$$topo.$$

Após a inserção:

$$topo \leftarrow topo + 1.$$

4.5 Operação desempilha

Para remover o elemento do topo, o índice é decrementado:

$$topo \leftarrow topo - 1.$$

Depois disso, o valor localizado nessa posição é retornado.

Esse mecanismo garante o comportamento LIFO.

4.6 Testes em Python

```
1 def executar_testes():
2
3     pilha = Pilha(3, int)
4
5     assert pilha.pilha_esta_vazia()
```

```

6
7     pilha.empilha(10)
8     pilha.empilha(20)
9
10    assert pilha.tamanho() == 2
11
12    assert pilha.desempilha() == 20
13    assert pilha.desempilha() == 10
14
15    assert pilha.pilha_esta_vazia()
16
17    pilha.empilha(1)
18    pilha.empilha(2)
19    pilha.empilha(3)
20
21    try:
22        pilha.empilha(4)
23        assert False
24
25    except PilhaCheiaErro:
26        pass
27
28    try:
29        pilha.empilha("texto")
30        assert False
31
32    except TipoErro:
33        pass
34
35    print("Todos os testes passaram.")
36
37
38 executar_testes()

```

Listing 4.2: Testes da implementação Python.

4.7 Resultado

Os testes verificam:

- criação da pilha;

- inserção;
- remoção;
- comportamento LIFO;
- tamanho;
- pilha cheia;
- tipo incompatível.

Capítulo 5

Implementação em C/C++

5.1 Descrição

A implementação em C++ utiliza uma classe para representar a pilha e um array para realizar o armazenamento interno.

Diferentemente de Python e JavaScript, o C++ permite maior controle sobre a memória utilizada pela estrutura.

A classe controla explicitamente a quantidade de elementos por meio do atributo `topo`.

5.2 Implementação

```
1 #include <iostream>
2 #include <stdexcept>
3
4 using namespace std;
5
6
7 class PilhaCheiaErro : public runtime_error {
8
9 public:
10
11     PilhaCheiaErro()
12         : runtime_error("A pilha esta cheia.") {}
13 };
14
15
```

```

16 class TipoErro : public runtime_error {
17
18 public:
19
20     TipoErro()
21         : runtime_error("Tipo de dado invalido.") {}
22 };
23
24
25 class Pilha {
26
27 private:
28
29     int capacidade;
30     int topo;
31     int* dados;
32
33 public:
34
35     Pilha(int capacidade)
36         : capacidade(capacidade), topo(0) {
37
38         if (capacidade <= 0) {
39
40             throw invalid_argument(
41                 "A capacidade deve ser positiva."
42             );
43         }
44
45         dados = new int[capacidade];
46     }
47
48
49     ~Pilha() {
50
51         delete[] dados;
52     }
53
54
55     bool pilha_esta_vazia() const {
56

```

```

57     return topo == 0;
58 }
59
60
61 bool pilha_esta_cheia() const {
62
63     return topo == capacidade;
64 }
65
66
67 int tamanho() const {
68
69     return topo;
70 }
71
72
73 void empilha(int dado) {
74
75     if (pilha_esta_cheia()) {
76
77         throw PilhaCheiaErro();
78     }
79
80     dados[topo] = dado;
81
82     topo++;
83 }
84
85
86 int desempilha() {
87
88     if (pilha_esta_vazia()) {
89
90         throw underflow_error(
91             "A pilha esta vazia."
92         );
93     }
94
95     topo--;
96
97     return dados[topo];

```

```

98     }
99
100
101     void troca() {
102
103         if (topo < 2) {
104
105             throw underflow_error(
106                 "Elementos insuficientes."
107             );
108         }
109
110         int aux = dados[topo - 1];
111
112         dados[topo - 1] =
113             dados[topo - 2];
114
115         dados[topo - 2] =
116             aux;
117     }
118 };
119
120
121 int main() {
122
123     try {
124
125         Pilha p(3);
126
127         p.empilha(10);
128         p.empilha(20);
129         p.empilha(30);
130
131         cout << p.desempilha()
132             << endl;
133
134         cout << p.desempilha()
135             << endl;
136
137     }
138

```

```

139     catch (const exception& erro) {
140
141         cerr << "Erro: "
142             << erro.what()
143             << endl;
144     }
145
146     return 0;
147 }

```

Listing 5.1: Classe Pilha implementada em C++.

5.3 Array

O atributo:

```

1 int* dados;

```

representa o armazenamento dos elementos.

A memória é alocada no construtor:

```

1 dados = new int[capacidade];

```

A quantidade de elementos é controlada pelo atributo `topo`.

5.4 Destruitor

Como a memória foi alocada manualmente, é necessário liberá-la.

Para isso, a classe possui:

```

1 ~Pilha() {
2     delete[] dados;
3 }

```

Esse procedimento evita que a memória permaneça alocada após a destruição do objeto.

5.5 Operação empilha

A inserção verifica inicialmente se a capacidade máxima foi atingida.

Se:

$$topo = capacidade,$$

a operação gera `PilhaCheiaErro`.

Caso contrário:

$$dados[topo] = dado$$

e depois:

$$topo = topo + 1.$$

5.6 Operação desempilha

A remoção verifica inicialmente se a pilha está vazia.

Se:

$$topo = 0,$$

não existe elemento disponível.

Caso contrário:

$$topo = topo - 1$$

e o elemento é retornado.

5.7 Testes em C++

```
1 #include <cassert>
2 #include <iostream>
3
4 using namespace std;
5
6
7 int main() {
8
```

```

9      Pilha p(3);
10
11      assert(
12          p.pilha_esta_vazia()
13      );
14
15      p.empilha(10);
16      p.empilha(20);
17
18      assert(
19          p.tamanho() == 2
20      );
21
22      assert(
23          p.desempilha() == 20
24      );
25
26      assert(
27          p.desempilha() == 10
28      );
29
30      assert(
31          p.pilha_esta_vazia()
32      );
33
34
35      p.empilha(1);
36      p.empilha(2);
37      p.empilha(3);
38
39
40      try {
41
42          p.empilha(4);
43
44          assert(false);
45
46      }
47
48      catch (const PilhaCheiaErro&) {
49

```

```
50         cout
51             << "Teste de pilha cheia: OK"
52             << endl;
53     }
54
55
56     cout
57         << "Todos os testes passaram."
58         << endl;
59
60
61     return 0;
62 }
```

Listing 5.2: Testes da implementação em C++.

Capítulo 6

Implementação em JavaScript

6.1 Descrição

JavaScript possui estruturas denominadas *TypedArrays*, que permitem representar dados numéricos utilizando tipos específicos.

Para a implementação da pilha foram considerados:

- `Int32Array` para inteiros;
- `Float64Array` para números de ponto flutuante;
- `Uint16Array` para representar caracteres.

6.2 Implementação

```
1 class PilhaCheiaErro extends Error {  
2  
3     constructor() {  
4  
5         super("A pilha esta cheia.");  
6  
7         this.name =  
8             "PilhaCheiaErro";  
9     }  
10 }  
11  
12  
13 class TipoErro extends Error {  
14
```

```

15     constructor() {
16
17         super("Tipo de dado invalido.");
18
19         this.name =
20             "TipoErro";
21     }
22 }
23
24
25 class Pilha {
26
27     constructor(capacidade, tipo) {
28
29         if (capacidade <= 0) {
30
31             throw new Error(
32                 "A capacidade deve ser positiva."
33             );
34         }
35
36         this.capacidade =
37             capacidade;
38
39         this.tipo =
40             tipo;
41
42         this.topo = 0;
43
44
45         if (tipo === "int") {
46
47             this.dados =
48                 new Int32Array(
49                     capacidade
50                 );
51         }
52
53         else if (tipo === "float") {
54
55             this.dados =

```

```

56         new Float64Array(
57             capacidade
58         );
59     }
60
61     else if (tipo === "char") {
62
63         this.dados =
64             new Uint16Array(
65                 capacidade
66             );
67     }
68
69     else {
70
71         throw new TipoErro();
72     }
73 }
74
75
76 pilha_esta_vazia() {
77
78     return this.topo === 0;
79 }
80
81
82 pilha_esta_cheia() {
83
84     return (
85         this.topo ===
86         this.capacidade
87     );
88 }
89
90
91 tamanho() {
92
93     return this.topo;
94 }
95
96

```

```

97     empilha(dado) {
98
99         if (
100             this.pilha_esta_cheia()
101         ) {
102
103             throw new PilhaCheiaErro();
104         }
105
106
107         if (this.tipo === "int") {
108
109             if (
110                 !Number.isInteger(dado)
111             ) {
112
113                 throw new TipoErro();
114             }
115         }
116
117
118         else if (
119             this.tipo === "float"
120         ) {
121
122             if (
123                 typeof dado !==
124                 "number"
125             ) {
126
127                 throw new TipoErro();
128             }
129         }
130
131
132         else if (
133             this.tipo === "char"
134         ) {
135
136             if (
137                 typeof dado !==

```

```

138         "string" ||
139         dado.length != 1
140     ) {
141
142         throw new TipoErro();
143     }
144
145     dado =
146         dado.charCodeAt(0);
147 }
148
149
150     this.dados[this.topo] =
151         dado;
152
153     this.topo++;
154 }
155
156
157 desempilha() {
158
159     if (
160         this.pilha_esta_vazia()
161     ) {
162
163         throw new Error(
164             "A pilha esta vazia."
165         );
166     }
167
168     this.topo--;
169
170     return this.dados[
171         this.topo
172     ];
173 }
174
175
176 troca() {
177
178     if (this.topo < 2) {

```

```

179
180         throw new Error(
181             "Elementos insuficientes."
182         );
183     }
184
185     const aux =
186         this.dados[
187             this.topo - 1
188         ];
189
190     this.dados[
191         this.topo - 1
192     ] =
193         this.dados[
194             this.topo - 2
195         ];
196
197     this.dados[
198         this.topo - 2
199     ] =
200         aux;
201 }
202 }

```

Listing 6.1: Classe Pilha implementada em JavaScript.

6.3 TypedArray

O `Int32Array` permite armazenar inteiros utilizando uma estrutura tipada.

Para valores de ponto flutuante é utilizado:

```
1 Float64Array
```

Para caracteres, o código utiliza:

```
1 Uint16Array
```

Como essa estrutura armazena valores numéricos, o caractere é convertido para seu código Unicode.

6.4 Verificação de tipos

A implementação verifica o tipo do elemento antes da inserção.

Para inteiros:

```
1 Number.isInteger(dado)
```

Para números:

```
1 typeof dado === "number"
```

Para caracteres é necessário verificar se o valor é uma string com exatamente um caractere.

6.5 Testes em JavaScript

```
1 function testarPilha() {  
2  
3     const pilha =  
4         new Pilha(3, "int");  
5  
6  
7     console.assert(  
8         pilha.pilha_esta_vazia()  
9     );  
10  
11  
12     pilha.empilha(10);  
13     pilha.empilha(20);  
14  
15  
16     console.assert(  
17         pilha.tamanho() === 2  
18     );  
19  
20  
21     console.assert(  
22         pilha.desempilha() === 20  
23     );  
24  
25  
26     console.assert(  

```

```
27     pilha.desempilha() === 10
28 );
29
30
31 console.assert(
32     pilha.pilha_esta_vazia()
33 );
34
35
36 pilha.empilha(1);
37 pilha.empilha(2);
38 pilha.empilha(3);
39
40
41 try {
42
43     pilha.empilha(4);
44
45     console.assert(false);
46
47 }
48
49 catch (erro) {
50
51     console.log(
52         "Teste de pilha cheia: OK"
53     );
54 }
55
56
57 try {
58
59     pilha.empilha(
60         "texto"
61     );
62
63     console.assert(false);
64
65 }
66
67 catch (erro) {
```

```
68
69     console.log(
70         "Teste de tipo: OK"
71     );
72 }
73
74
75     console.log(
76         "Todos os testes passaram."
77     );
78 }
79
80
81 testarPilha();
```

Listing 6.2: Testes da implementação JavaScript.

Capítulo 7

Testes e Debug

7.1 Estratégia de testes

Os testes foram desenvolvidos para verificar tanto as operações normais quanto situações que devem produzir erros.

Os principais casos são apresentados na Tabela 7.1.

Caso	Resultado esperado
Criar pilha	A pilha deve iniciar vazia.
Inserir elemento válido	Elemento armazenado no topo.
Remover elemento	Retorna o último elemento inserido.
Consultar tamanho	Retorna a quantidade correta.
Inserir quando cheia	Gera <code>PilhaCheiaErro</code> .
Inserir tipo incompatível	Gera <code>TipoErro</code> .
Remover quando vazia	Gera erro.
Trocar elementos	Inverte os dois elementos do topo.

Tabela 7.1: Casos de teste.

7.2 Teste do comportamento LIFO

O principal teste funcional consiste em inserir:

10, 20, 30.

A pilha será:

[10, 20, 30].

As remoções devem produzir:

30, 20, 10.

Caso a ordem fosse:

10, 20, 30,

a estrutura não estaria funcionando como uma pilha.

7.3 Debug

Durante o debug, o principal elemento observado é o índice `topo`.

Inicialmente:

$$topo = 0.$$

Após inserir o primeiro elemento:

$$topo = 1.$$

Após inserir o segundo:

$$topo = 2.$$

Após remover o elemento:

$$topo = 1.$$

Esse controle é fundamental para impedir acesso a posições inválidas.

7.4 Teste de pilha cheia

Considere uma pilha de capacidade:

$$C = 3.$$

Após inserir:

10, 20, 30,

temos:

$topo = 3.$

Como:

$topo = C,$

a pilha está cheia.

Uma nova tentativa:

empilha(40)

deve produzir:

PilhaCheiaErro.

7.5 Teste de tipo

Para uma pilha de inteiros:

Pilha(5, `int`),

a inserção de:

10

é válida.

Porém:

"dez

deve ser rejeitado.

Esse teste garante a homogeneidade dos elementos.

Capítulo 8

Code Review

8.1 Código inicial

A Inteligência Artificial foi utilizada como ferramenta de auxílio durante o desenvolvimento.

A primeira versão serviu como ponto de partida para a implementação. Entretanto, após a análise do código, foram identificadas situações que precisavam ser modificadas para atender integralmente à especificação.

8.2 Problemas identificados

Os principais pontos analisados foram:

- utilização de estruturas dinâmicas;
- ausência de algumas verificações de tipo;
- tratamento incompleto de situações de erro;
- necessidade de controle explícito da capacidade;
- necessidade de testes adicionais;
- diferenças entre os recursos disponíveis em cada linguagem.

8.3 Uso inadequado de lista

Uma solução inicial em Python poderia utilizar:

```
1 self.dados = []
```

e inserir elementos utilizando:

```
1 self.dados.append(dado)
```

Embora essa solução represente uma pilha funcional, ela não reproduz adequadamente o requisito de utilizar uma estrutura baseada em array com capacidade definida.

Por isso, a implementação foi modificada para utilizar:

```
1 from array import array
```

8.4 Verificação de tipos

Outro ponto importante foi a necessidade de garantir que a pilha não receba elementos de tipos diferentes.

O código final realiza uma verificação antes da inserção.

Essa alteração foi necessária porque uma implementação que simplesmente armazena qualquer objeto não atende ao requisito de homogeneidade.

8.5 Importância do Code Review

O Code Review mostrou que código gerado por Inteligência Artificial não deve ser utilizado sem análise.

A IA pode produzir uma implementação que funciona em exemplos simples, mas que não necessariamente atende exatamente ao enunciado.

A revisão permitiu verificar:

- requisitos;
- tipos;
- capacidade;
- tratamento de erros;
- complexidade;
- qualidade dos testes.

Capítulo 9

Análise de Complexidade

9.1 Inserção

A operação `empilha()` acessa diretamente a posição indicada pelo topo.

Portanto:

$$T(n) = O(1).$$

9.2 Remoção

A operação `desempilha()` também acessa diretamente a posição do topo.

Assim:

$$T(n) = O(1).$$

9.3 Verificação de vazio

A operação verifica apenas:

$$topo = 0.$$

Logo:

$$T(n) = O(1).$$

9.4 Verificação de cheio

A condição analisada é:

$$topo = capacidade.$$

Portanto:

$$T(n) = O(1).$$

9.5 Tamanho

Como a quantidade de elementos é armazenada no atributo `topo`, o tamanho pode ser retornado diretamente.

Assim:

$$T(n) = O(1).$$

9.6 Troca

A operação `troca()` altera somente dois elementos.

Consequentemente:

$$T(n) = O(1).$$

9.7 Complexidade espacial

Se a capacidade da pilha for n , o array precisa reservar espaço para n elementos.

Portanto:

$$S(n) = O(n).$$

9.8 Resumo

Operação	Complexidade
empilha	$O(1)$
desempilha	$O(1)$
pilha_esta_vazia	$O(1)$
pilha_esta_cheia	$O(1)$
tamanho	$O(1)$
troca	$O(1)$

Tabela 9.1: Complexidade das operações.

Capítulo 10

Comparação entre as Implementações

As três implementações possuem a mesma estrutura lógica, porém utilizam mecanismos diferentes de acordo com as características de cada linguagem.

	Python	C/C++	JavaScript
Armazenamento	<code>array</code>	Array	TypedArray
Tipagem	Verificada em execução	Estática	Verificada em execução
Memória	Gerenciada automaticamente	Controle explícito	Gerenciada automaticamente
Exceções	Suportadas	Suportadas	Suportadas
Capacidade	Controlada pela classe	Controlada pela classe	Controlada pela classe

Tabela 10.1: Comparação das implementações.

10.1 Python

Python apresenta uma implementação relativamente simples e possui recursos que facilitam o tratamento de exceções e a verificação dos tipos.

O módulo `array` permite utilizar armazenamento tipado.

10.2 C/C++

C++ oferece maior controle sobre memória e tipos.

Essa característica permite uma implementação mais próxima do funcionamento de estruturas de baixo nível, porém exige maior atenção ao gerenciamento da memória.

10.3 JavaScript

JavaScript possui tipagem dinâmica, mas os *TypedArrays* permitem utilizar estruturas específicas para dados numéricos.

A implementação também precisa realizar verificações explícitas para garantir que os dados inseridos sejam compatíveis com o tipo definido.

Capítulo 11

Testes de Estresse

Os testes de estresse têm como objetivo verificar o comportamento da estrutura quando são realizadas muitas operações consecutivas.

Considere uma pilha com capacidade N .

Os elementos:

$$1, 2, 3, \dots, N$$

são inseridos sequencialmente.

As remoções devem ocorrer na ordem:

$$N, N - 1, N - 2, \dots, 1.$$

Esse teste verifica simultaneamente:

- controle do topo;
- limite de capacidade;
- comportamento LIFO;
- acesso ao array;
- consistência após várias operações.

Também é importante realizar sequências alternadas:

$$\text{empilha} \rightarrow \text{desempilha} \rightarrow \text{empilha} \rightarrow \text{desempilha}.$$

Isso permite verificar se o valor do topo permanece correto durante toda a execução.

Capítulo 12

Post-Mortem

12.1 Objetivo

O Post-Mortem apresenta uma análise do processo de desenvolvimento, registrando decisões, dificuldades, correções e aprendizados.

12.2 Uso da Inteligência Artificial

A Inteligência Artificial foi utilizada como ferramenta de apoio para gerar uma primeira estrutura de código e auxiliar durante o desenvolvimento.

A utilização da IA permitiu acelerar a criação das primeiras versões, mas as respostas geradas precisaram ser analisadas.

12.3 Problemas encontrados

Um dos principais problemas foi a diferença entre produzir um código que funciona e produzir um código que atende exatamente à especificação.

Entre os problemas observados estão:

- utilização de estruturas inadequadas;
- verificações de tipo incompletas;
- necessidade de tratamento de exceções;
- diferenças entre as linguagens;
- necessidade de ampliar os testes.

12.4 Correções realizadas

Após o Code Review, foram realizadas alterações para:

- controlar explicitamente a capacidade;
- utilizar arrays adequados;
- verificar os tipos;
- tratar erros;
- testar situações normais;
- testar situações excepcionais;
- verificar o comportamento LIFO.

12.5 Aprendizados

O desenvolvimento permitiu compreender melhor a implementação de uma estrutura de dados utilizando diferentes paradigmas e recursos de linguagem.

Também foi possível perceber que a utilização de ferramentas de IA exige conhecimento suficiente para avaliar o código produzido.

A ferramenta pode auxiliar na programação, mas a responsabilidade pela correção da implementação continua sendo do programador.

12.6 Resultado final

Ao final do processo, as três implementações passaram a possuir a mesma finalidade e seguir os mesmos princípios:

- capacidade limitada;
- comportamento LIFO;
- armazenamento em array;
- controle do topo;
- tratamento de erros;
- testes funcionais.

Capítulo 13

Conclusão

O desenvolvimento da classe **Pilha** permitiu estudar praticamente uma das estruturas de dados lineares fundamentais da computação.

A partir da implementação em Python, C/C++ e JavaScript foi possível observar que uma mesma estrutura abstrata pode ser implementada de formas diferentes de acordo com os recursos de cada linguagem.

As três implementações seguem o princípio LIFO, no qual o último elemento inserido é o primeiro a ser removido.

Também foram implementados mecanismos de controle da capacidade, verificação de tipos, tratamento de erros e operações básicas da estrutura.

Os testes realizados permitiram verificar o comportamento esperado tanto em situações normais quanto em situações de erro.

A análise de complexidade demonstrou que as principais operações da pilha apresentam complexidade temporal:

$$O(1).$$

O espaço necessário para armazenar a estrutura é:

$$O(n).$$

Outro ponto importante foi o processo de desenvolvimento assistido por Inteligência Artificial. A experiência demonstrou que uma ferramenta de IA pode ser útil para gerar uma primeira solução e auxiliar na programação, mas o código precisa passar por revisão, testes e validação.

Portanto, o trabalho possibilitou não apenas implementar uma pilha, mas também

desenvolver conhecimentos relacionados à programação orientada a objetos, estruturas de dados, tratamento de erros, complexidade, testes, debug, Code Review e uso responsável de Inteligência Artificial.